1. Implement the data link layer farming methods such as character stuffing and bit stuffing.

```
#include <stdio.h>
#include <string.h>

#define FLAG '~'
#define ESC '}'

void charStuff(char data[]) {
   printf("\nStuffed Frame: %c", FLAG);

   for (int i = 0; data[i] != '\0'; i++) {
      if (data[i] == FLAG || data[i] == ESC)
         printf("%c", ESC);
      printf("%c", data[i]);
   }

   printf("%c\n", FLAG);
}

void bitStuff(char bits[]) {
   int count = 0;

   printf("\nStuffed Frame: 01111110");

   for (int i = 0; bits[i] != '\0'; i++) {
      printf("%c", bits[i]);

      if (bits[i] == '1') {
         count++;
         if (count == 5) {
```

```
            printf("0");
            count = 0;
         }
      } else {
         count = 0;
      }
   }

   printf("01111110\n");
}

int main() {
   int ch;
   char data[100], bits[100];

   printf("1. Character Stuffing\n");
   printf("2. Bit Stuffing\n");
   printf("Enter Choice: ");
   scanf("%d", &ch);

   switch (ch) {
      case 1:
         printf("Enter Data: ");
         scanf("%s", data);
         charStuff(data);
         break;

      case 2:
         printf("Enter Bit Stream: ");
         scanf("%s", bits);
         bitStuff(bits);
         break;

      default:
         printf("Invalid Choice\n");
```

```
    }

    return 0;
}
```

Output:

1. Character Stuffing
2. Bit Stuffing
Enter Choice: 1
Enter Data: AB~CD}EF

Stuffed Frame: ~AB}~CD}}EF~

1. Character Stuffing
2. Bit Stuffing
Enter Choice: 2
Enter Bit Stream: 1111101111110

Stuffed Frame: 011111101111100111111010011111110

2. Write a C program to develop a DNS client server to resolve the given hostname.

```
// dns_client.c
#include <stdio.h>
#include <stdlib.h>
#include <netdb.h>
#include <string.h>
#include <arpa/inet.h>
int main() {
char hostname[100];
struct hostent *host_entry;
char *IPbuffer;
// Get hostname input
printf("Enter hostname to resolve (e.g., www.google.com): ");
scanf("%s", hostname);
// Resolve hostname
host_entry = gethostbyname(hostname);
if (host_entry == NULL) {
fprintf(stderr, "Could not resolve hostname.\n");
return 1;
}
// Convert to IP string
IPbuffer = inet_ntoa(*((struct in_addr*)host_entry->h_addr_list[0]));
printf("Host name : %s\n", host_entry->h_name);
printf("IP Address : %s\n", IPbuffer);
return 0;
}
```

OUTPUT:

```
Enter hostname to resolve (e.g., www.google.com): www.google.com
Host name : www.google.com
IP Address : 142.251.156.119
```

3. Implement on a data set of characters the three CRC polynomials – CRC-12, CRC- 16and CRC- CCIP.

```
#include <stdio.h>
#include <string.h>

unsigned int calculateCRC(char *data, unsigned int poly, int crcBits)
{
   unsigned int crc = 0;
   int i, j;

   for(i = 0; i < strlen(data); i++)
   {
      crc ^= ((unsigned int)data[i] << (crcBits - 8));

      for(j = 0; j < 8; j++)
      {
         if(crc & (1 << (crcBits - 1)))
            crc = (crc << 1) ^ poly;
         else
            crc <<= 1;
      }
   }

   return crc & ((1 << crcBits) - 1);
}

int main()
{
   char data[100];

   printf("Enter character data: ");
   scanf("%s", data);
```

```
    printf("\nCRC Values:\n");
    printf("CRC-12   : %03X\n", calculateCRC(data, 0x80F, 12));
    printf("CRC-16   : %04X\n", calculateCRC(data, 0x8005, 16));
    printf("CRC-CCITT: %04X\n", calculateCRC(data, 0x1021, 16));

    return 0;
}
```

OUTPUT:

```
Enter character data: HELLO

CRC Values:
CRC-12 : B9C
CRC-16 : E141
CRC-CCITT: 58DA
```

4. Implement Dijkstra's algorithm to compute the shortest path in a graph.

```
#include <stdio.h>

#define V 5
#define INF 99999

// Function to find the vertex with minimum distance
int minDistance(int dist[], int visited[]) {
    int min = INF, min_index = -1;

    for (int v = 0; v < V; v++) {
        if (!visited[v] && dist[v] < min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}
```

```
// Dijkstra's Algorithm
void dijkstra(int graph[V][V], int src) {
    int dist[V];
    int visited[V];

    for (int i = 0; i < V; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }

    dist[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited);
        visited[u] = 1;

        for (int v = 0; v < V; v++) {
            if (!visited[v] &&
                graph[u][v] &&
                dist[u] != INF &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Vertex\tDistance from Source\n");
    for (int i = 0; i < V; i++) {
        printf("%d\t%d\n", i, dist[i]);
    }
}

int main() {
```

```
    int graph[V][V] = {
        {0, 10, 0, 5, 0},
        {0, 0, 1, 2, 0},
        {0, 0, 0, 0, 4},
        {0, 3, 9, 0, 2},
        {7, 0, 6, 0, 0}
    };
    int source = 0;

    dijkstra(graph, source);

    return 0;
}
```

OUTPUT

```
Vertex  Distance from Source
0       0
1       8
2       9
3       5
4       7
```

5. Take an example subnet graph with weights indicating delay between nodes. Now obtain Routing table art each node using distance vector routing algorithm

```
#include <stdio.h>

#define N 4
#define INF 999

int main()
{
   int cost[N][N] = {
      {0, 1, 3, 7},
      {1, 0, 1, INF},
      {3, 1, 0, 2},
      {7, INF, 2, 0}
   };
   int dist[N][N];
   int nextHop[N][N];
   int i, j, k;
    // Initialize distance and next hop tables
   for(i = 0; i < N; i++)
   {
      for(j = 0; j < N; j++)
      {
         dist[i][j] = cost[i][j];

         if(i == j)
            nextHop[i][j] = i;
         else if(cost[i][j] != INF)
            nextHop[i][j] = j;
         else
            nextHop[i][j] = -1;
      }
```

```
    }
    // Distance Vector Algorithm
    for(k = 0; k < N; k++)
    {
        for(i = 0; i < N; i++)
        {
            for(j = 0; j < N; j++)
            {
                if(dist[i][j] > dist[i][k] + dist[k][j])
                {
                    dist[i][j] = dist[i][k] + dist[k][j];
                    nextHop[i][j] = nextHop[i][k];
                }
            }
        }
    }

    // Print Routing Tables
    for(i = 0; i < N; i++)
    {
        printf("\nRouting Table for Node %c\n", 'A' + i);
        printf("Destination\tNext Hop\tCost\n");

        for(j = 0; j < N; j++)
        {
            printf("%c\t\t%c\t\t%d\n",
                   'A' + j,
                   (nextHop[i][j] == -1) ? '-' : ('A' + nextHop[i][j]),
                   dist[i][j]);
        }
    }

    return 0;
}
```

OUTPUT:

Routing Table for Node A

Destination	Next Hop	Cost
A	A	0
B	B	1
C	B	2
D	B	4

Routing Table for Node B

Destination	Next Hop	Cost
A	A	1
B	B	0
C	C	1
D	C	3

Routing Table for Node C

Destination	Next Hop	Cost
A	B	2
B	B	1
C	C	0
D	D	2

Routing Table for Node D

Destination	Next Hop	Cost
A	C	4
B	C	3
C	C	2
D	D	0

6. Write a C program to Take an example subnet of hosts. Obtain broadcast tree for it.

```
#include <stdio.h>
#define MAX 10
int main() {
    int n, i, j;
    int graph[MAX][MAX];
    int visited[MAX] = {0};
    int queue[MAX], front = 0, rear = 0;
    int source;
    printf("Enter number of hosts in subnet: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    printf("Enter source host (0 to %d): ", n - 1);
    scanf("%d", &source);
    visited[source] = 1;
```

```
    queue[rear++] = source;
     printf("\nBroadcast Tree:\n");
     while(front < rear) {
        int u = queue[front++];
      for(i = 0; i < n; i++) {
            if(graph[u][i] == 1 && !visited[i]) {
                visited[i] = 1;
                queue[rear++] = i;


                printf("%d --> %d\n", u, i);
            }
        }
    }

    return 0;
}
```

INPUT:

Enter number of hosts in subnet: 5

Enter adjacency matrix:

0 1 1 0 0

1 0 1 1 0

1 1 0 0 1

0 1 0 0 1

0 0 1 1 0

Enter source host (0 to 4): 0

OUTPUT:

Broadcast Tree:

0 --> 1

0 --> 2

1 --> 3

2 --> 4

7. Write a client-server application for chat using UDP

Server Program (server.c)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    char buffer[BUFFER_SIZE];
    struct sockaddr_in server_addr, client_addr;
    socklen_t len = sizeof(client_addr);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
    server_addr.sin_addr.s_addr = INADDR_ANY;
```

```
bind(sockfd, (struct sockaddr *)&server_addr, sizeof(server_addr));

printf("UDP Chat Server Started...\n");

while (1) {
    memset(buffer, 0, BUFFER_SIZE);

    recvfrom(sockfd, buffer, BUFFER_SIZE, 0,
            (struct sockaddr *)&client_addr, &len);

    printf("Client: %s\n", buffer);

    printf("Server: ");
    fgets(buffer, BUFFER_SIZE, stdin);

    sendto(sockfd, buffer, strlen(buffer), 0,
          (struct sockaddr *)&client_addr, len);

    if (strncmp(buffer, "bye", 3) == 0)
        break;
}
```

```
    close(sockfd);
    return 0;
}
```

Client Program (client.c)

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int sockfd;
    char buffer[BUFFER_SIZE];
    struct sockaddr_in server_addr;
    socklen_t len = sizeof(server_addr);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(PORT);
```

```
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

printf("UDP Chat Client Started...\n");

while (1) {
    printf("Client: ");
    fgets(buffer, BUFFER_SIZE, stdin);

    sendto(sockfd, buffer, strlen(buffer), 0,
           (struct sockaddr *)&server_addr, len);

    if (strncmp(buffer, "bye", 3) == 0)
        break;

    memset(buffer, 0, BUFFER_SIZE);

    recvfrom(sockfd, buffer, BUFFER_SIZE, 0,
             (struct sockaddr *)&server_addr, &len);

    printf("Server: %s\n", buffer);
}

close(sockfd);
```

```
    return 0;
}
```

OUTPUT:

Server Side

UDP Chat Server Started...

Client: Hello Server

Server: Hello Client

Client: How are you?

Server: I am fine.

Client: bye

Client Side

UDP Chat Client Started...

Client: Hello Server

Server: Hello Client

Client: How are you?

Server: I am fine.

Client: bye

8. Implement programs using raw sockets (like packet capturing and filtering)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <netinet/ip.h>

int main()
{
    int sockfd;
    char buffer[65536];

    struct sockaddr_in saddr;
    socklen_t saddr_size = sizeof(saddr);

    // Create raw socket for ICMP packets
    sockfd = socket(AF_INET, SOCK_RAW, IPPROTO_ICMP);

    if (sockfd < 0)
    {
        perror("socket");
```

```
    return 1;
}

printf("Capturing ICMP packets...\n");

while (1)
{
    ssize_t data_size = recvfrom(sockfd, buffer, sizeof(buffer), 0,
                                 (struct sockaddr *)&saddr,
                                 &saddr_size);

    if (data_size < 0)
    {
        perror("recvfrom");
        close(sockfd);
        return 1;
    }

    struct iphdr *ip = (struct iphdr *)buffer;

    struct in_addr src, dst;
    src.s_addr = ip->saddr;
    dst.s_addr = ip->daddr;
```

```
        printf("\n===== ICMP Packet Captured =====\n");
        printf("Source IP      : %s\n", inet_ntoa(src));
        printf("Destination IP : %s\n", inet_ntoa(dst));
        printf("Protocol       : %d\n", ip->protocol);
        printf("Packet Size    : %ld bytes\n", data_size);
    }   close(sockfd);
    return 0;
}
```

OUTPUT:

```
Capturing ICMP packets...


===== ICMP Packet Captured =====
Source IP      : 192.168.1.5
Destination IP : 8.8.8.8
Protocol       : 1
Packet Size    : 84 bytes
===== ICMP Packet Captured =====
Source IP      : 8.8.8.8
Destination IP : 192.168.1.5
Protocol       : 1
Packet Size    : 84 bytes
```

9. Write a C program to perform sliding window protocol.

```
#include <stdio.h>

int main()
{
   int frames, window, i, j;

   printf("Enter the number of frames: ");
   scanf("%d", &frames);

   printf("Enter the window size: ");
   scanf("%d", &window);

   printf("\nTransmission Process:\n");

   for(i = 1; i <= frames; i += window)
   {
      printf("\nSending frames: ");

      for(j = i; j < i + window && j <= frames; j++)
      {
         printf("%d ", j);
```

```
        }
        printf("\nAcknowledgment received for frames: ");
        for(j = i; j < i + window && j <= frames; j++)
        {
            printf("%d ", j);
        }  printf("\n");
    }

    return 0;
}
```

OUTPUT:

Enter the number of frames: 8

Enter the window size: 3

Transmission Process:

Sending frames: 1 2 3

Acknowledgment received for frames: 1 2 3

Sending frames: 4 5 6

Acknowledgment received for frames: 4 5 6

Sending frames: 7 8

Acknowledgment received for frames: 7 8

10. Get the MAC or Physical address of the system using Address Resolution Protocol.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ifaddrs.h>
#include <sys/types.h>
#include <netpacket/packet.h>
#include <net/if.h>
#include <arpa/inet.h>
#include <netdb.h>
int main() {
struct ifaddrs *ifaddr, *ifa;
int family, n;
int found = 0;
if (getifaddrs(&ifaddr) == -1) {
perror("getifaddrs");
exit(EXIT_FAILURE);
}
/* Iterate through the linked list of interfaces */
for (ifa = ifaddr; ifa != NULL; ifa = ifa->ifa_next) {
if (ifa->ifa_addr == NULL)
```

```
continue;
family = ifa->ifa_addr->sa_family;
/* Check for AF_PACKET which contains MAC info */
if (family == AF_PACKET) {
struct sockaddr_ll *sll = (struct sockaddr_ll *) ifa->ifa_addr;
/* Skip loopback and ensure MAC exists */
if (sll->sll_halen == 6 && !(ifa->ifa_flags & IFF_LOOPBACK)) {
printf("Interface: %s\n", ifa->ifa_name);
printf("MAC Address: ");
for (n = 0; n < sll->sll_halen; n++) {
printf("%02x%c", sll->sll_addr[n],

(n + 1 == sll->sll_halen) ? '\n' : ':');
}
found = 1;
break; // stop after first valid MAC
}
}
}
freeifaddrs(ifaddr);
if (!found) {
printf("Could not find a non-loopback interface with a MAC address.\n");
return EXIT_FAILURE;
```

```
}

return EXIT_SUCCESS;

}
```

Output:

```
Interface: eth0

MAC Address: 12:9e:e6:19:ec:b7
```

11. Simulate the Implementing Routing Protocols using border gateway protocol (BGP)

```
#include <stdio.h>


struct Route
{
    int destination;
    int nextHopAS;
    int pathLength;
};


int main()
{
    int n, i, best = 0;


    printf("Enter number of available routes: ");
    scanf("%d", &n);


    struct Route routes[n];


    for(i = 0; i < n; i++)
    {
```

```
    printf("\nRoute %d\n", i + 1);

    printf("Destination Network: ");
    scanf("%d", &routes[i].destination);

    printf("Next Hop AS: ");
    scanf("%d", &routes[i].nextHopAS);

    printf("AS Path Length: ");
    scanf("%d", &routes[i].pathLength);
}

for(i = 1; i < n; i++)
{
    if(routes[i].pathLength < routes[best].pathLength)
    {
        best = i;
    }
}
printf("\n----- BGP Routing Table -----\n");
for(i = 0; i < n; i++)
{
    printf("Destination: %d\tNext Hop AS: %d\tPath Length: %d\n",
```

```
            routes[i].destination,
            routes[i].nextHopAS,
            routes[i].pathLength);
    }
    printf("\nBest Route Selected by BGP:\n");
    printf("Destination Network : %d\n", routes[best].destination);
    printf("Next Hop AS         : %d\n", routes[best].nextHopAS);
    printf("AS Path Length      : %d\n", routes[best].pathLength);

    return 0;
}
```

OUTPUT:

```
Enter number of available routes: 3
 Route 1
Destination Network: 100
Next Hop AS: 65001
AS Path Length: 5
 Route 2
Destination Network: 100
Next Hop AS: 65002
AS Path Length: 3
 Route 3
```

Destination Network: 100

Next Hop AS: 65003

AS Path Length: 4

----- BGP Routing Table -----

Destination: 100 Next Hop AS: 65001 Path Length: 5

Destination: 100 Next Hop AS: 65002 Path Length: 3

Destination: 100 Next Hop AS: 65003 Path Length: 4

Best Route Selected by BGP:

Destination Network : 100

Next Hop AS : 65002

AS Path Length : 3

12. Simulate the OPEN SHORTEST PATH FIRST routing protocol based on the cost assigned to the path.

```
#include <stdio.h>

#define MAX 20

#define INF 9999

void dijkstra(int graph[MAX][MAX], int n, int src)

{

   int dist[MAX], visited[MAX];

   int i, j, u, min;

   // Initialize distances and visited array

   for(i = 0; i < n; i++)

   {

      dist[i] = INF;

      visited[i] = 0;

   }


   dist[src] = 0;


   // Find shortest path for all vertices

   for(i = 0; i < n - 1; i++)

   {

      min = INF;
```

```
    for(j = 0; j < n; j++)
    {
        if(!visited[j] && dist[j] < min)
        {
            min = dist[j];
            u = j;
        }
    }

    visited[u] = 1;
    for(j = 0; j < n; j++)
    {
        if(!visited[j] &&
            graph[u][j] != INF &&
            dist[u] + graph[u][j] < dist[j])
        {
            dist[j] = dist[u] + graph[u][j];
        }
    }
}

printf("\nRouting Table for Router %d\n", src);
```

```
    printf("Destination\tCost\n");
    for(i = 0; i < n; i++)
    {
        printf("%d\t\t%d\n", i, dist[i]);
    }
}
int main()
{
    int n, src;
    int graph[MAX][MAX];
    int i, j;
    printf("Enter number of routers: ");
    scanf("%d", &n);
    printf("Enter cost matrix (9999 for no direct link):\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    } printf("Enter source router: ");
    scanf("%d", &src);
    dijkstra(graph, n, src);
```

```
    return 0;
}
```

INPUT:

Enter number of routers: 5

Enter cost matrix (9999 for no direct link):

0 2 9999 1 9999

2 0 3 2 9999

9999 3 0 4 5

1 2 4 0 3

9999 9999 5 3 0

Enter source router: 0

OUTPUT:

Routing Table for Router 0

Destination	Cost
0	0
1	2
2	5
3	1
4	4

13. Install Wireshark Tool on PC and use into:

a) Capture network traffic
b) Determine default gateway address of your network
c) Examine frame format and contents of Ethernet frames
d) Filter and examine only ICMP traffic
e) Run various network services like ping, ssh, dns ..etc and examine the traffic captured by Wire shark

Aim:

Install Wireshark Tool on PC and use into:

a) Capture network traffic
b) Determine default gateway address of your network
c) Examine frame format and contents of Ethernet frames
d) Filter and examine only ICMP traffic
e) Run various network services like ping, ssh, dns etc. and examine the traffic captured by Wire shark

Description:

Wireshark is an open-source network protocol analyzer used to capture and examine data packets traveling over a network. It helps in monitoring network traffic, analyzing performance issues, and understanding communication between devices in real-time.

Step-by-Step Procedure:

Step 1: Install Wireshark Tool on PC

1. Download Wireshark from the official website.
2. Run the installation setup.
3. Install WinPcap or Npcap when prompted.
4. Launch Wireshark after installation.

Step 2: Capture Network Traffic

1. Open Wireshark and select the active network interface.
2. Click Start Capturing Packets.
3. Perform any network activity such as opening a website or pinging a server.
4. Stop the capture after observing the traffic.

Step 3: Determine Default Gateway Address

1. Open Command Prompt or Terminal.
2. Type the command to view network configuration.
3. Note the Default Gateway IP Address.
4. Use a filter in Wireshark to view traffic related to the gateway.

Step 4: Examine Ethernet Frame Format

1. Locate a captured Ethernet II frame.
2. Expand Frame, Ethernet II, and Internet Protocol sections.
3. Observe source, destination, and type fields.

Step 5: Filter and Examine ICMP Traffic

1. In Wireshark, enter the ICMP filter.
2. Run a ping command.
3. Observe the captured ICMP packets.

Step 6: Run Network Services and Examine Traffic

Perform various services and observe packets in Wireshark.

Service	Command/Action	Filter
Ping	ping command	icmp
SSH	Connect via SSH	tcp.port == 22
DNS	DNS lookup	dns

HTTP	Open website	http
HTTPS	Access secure site	tls

14.Simulate a three-node point-to-point network with a duplex link between them. Set the queue size and vary the bandwidth and find the number of packets dropped.

STEPS:

Step1: Select the hub icon on the toolbar and drag it onto the working window.

Step2: Select the host icon on the toolbar and drag it onto the working window. Repeat this for another host icon.

Step3: Select the link icon on the toolbar and drag it on the screen from host (node 1) to the hub and again from host(node 2) to the hub. Here the hub acts as node 3 in the point-to-point network. This leads to the creation of the 3-node point-to-point network topology. Save this topology as a .tpl file.

Step4: Double-click on host(node 1), a host dialog box will open up. Click on Node editor and you can see the different layers- interface, ARP, FIFO, MAC, TCPDUMP, Physical layers. Select MAC and then select full-duplex for switches and routers and half duplex for hubs, and in log Statistics, select Number of Drop Packets, Number of Collisions, Throughput of incoming packets and Throughput of outgoing packets. Select FIFO and set the queue size to 50 and press OK. Then click on Add. Another dialog box pops up. Click on the Command box and type the Command according to the following syntax:
stg [-t duration(sec)] [-p port number]HostIPaddr and click OK.

Step 5: Double-click on host (node 2), and follow the same step as above with only change in command according to the following syntax: rtg [-t] [-w log] [-p port number] and click OK.

Step 6: Double click on the link between node 1 and the hub to set the bandwidth to some initial value say, 10 Mbps. Repeat the same for the other node.

Step 7: Click on the E button (Edit Property) present on the toolbar in order to save the changes made to the topology. Now click on the R button (RunSimulation). By doing so a user can run/pause/continue/stop/abort/disconnect/reconnect/submit a simulation. No simulation settings can be changed in this mode.

Step 8: Now go to Menu->Simulation->Run. Executing this command will submit he current simulation job to one available simulation server managed by the dispatcher. When the simulation server is executing, the user will see the time knot at the bottom of the screen move. The time knot reflects the current virtual time (progress) of the simulation case.

Step 9: To start the playback, the user can left-click the start icon(|>) of the time bar located at the bottom. The animation player will then start playing the recorded packet animation.

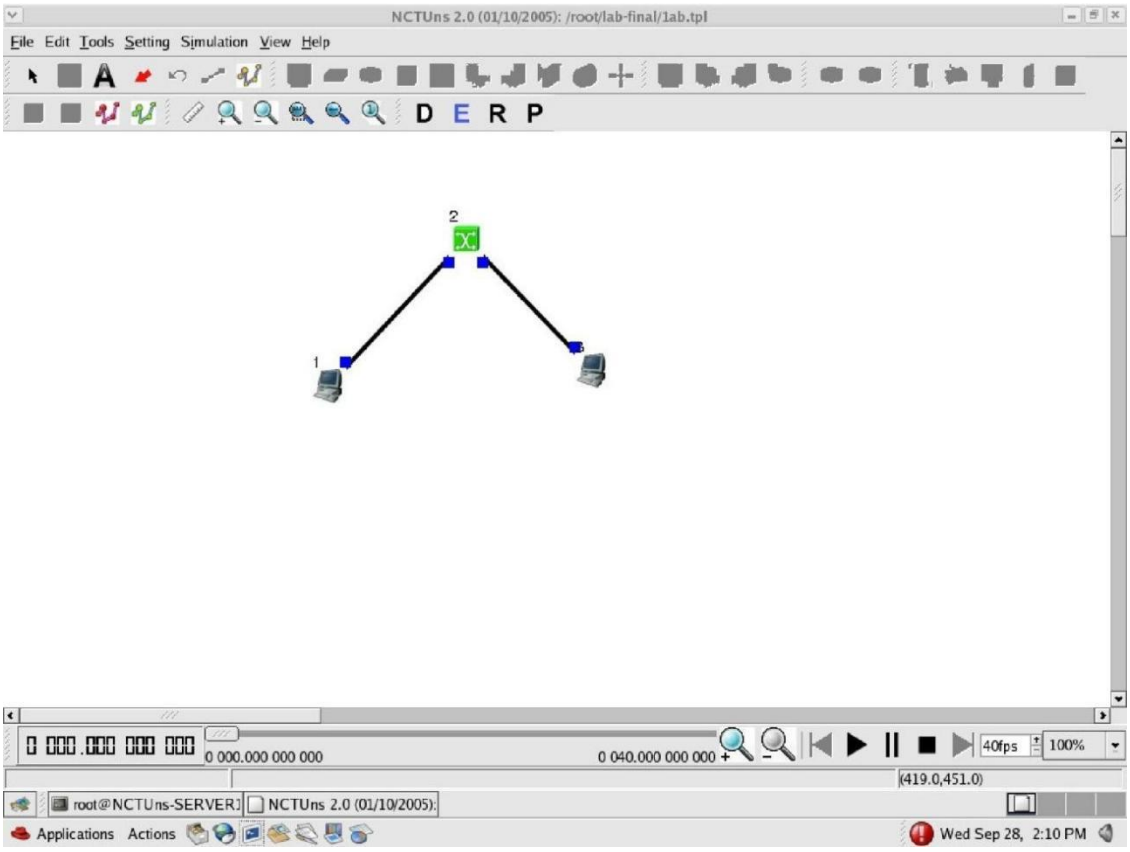
Step 10: Change the bandwidth say, 9 Mbps, and run the simulation and compare the two results.

Step 12: To view the results, go to the filename. results folder.

Note: To get the syntax of any command, double click on the host icon. Host dialog boxes appear and then choose App. Usage.

The screenshot below explain the topology.

OUTPUT:


NCTUns 2.0 (01/10/2005): /root/lab-final/1ab.tpl
File Edit Tools Setting Simulation View Help
D E R P
2
1
0 000.000 000 000
0 040.000 000 000
40fps
100%
(419.0,451.0)
root@NCTUns-SERVER:
NCTUns 2.0 (01/10/2005):
Applications Actions
Wed Sep 28, 2:10 PM

15 Simulate a four-node point-to-point network and connect the link as follows: Apply a TCP agent between n0 to n3 and apply a UDP agent between n1 and n3. Apply relevant applications over TCP and UDP agents changing the parameters and determine the number of packets sent by two agents.

STEPS:

Step 1: Create the topology as specified in the question, in the draw mode of th simulator.

Step 2: Go to edit mode and save the topology.

Step 3: Setup a TCP connection between node 1 and node 3 using the following commands:

stcp [-p port] [-l writesize]
hostIPaddr rtcp [-p port] [-l
readsize]

Step 4: Setup a UDP connection between node 2 and node 3 using the following commands:

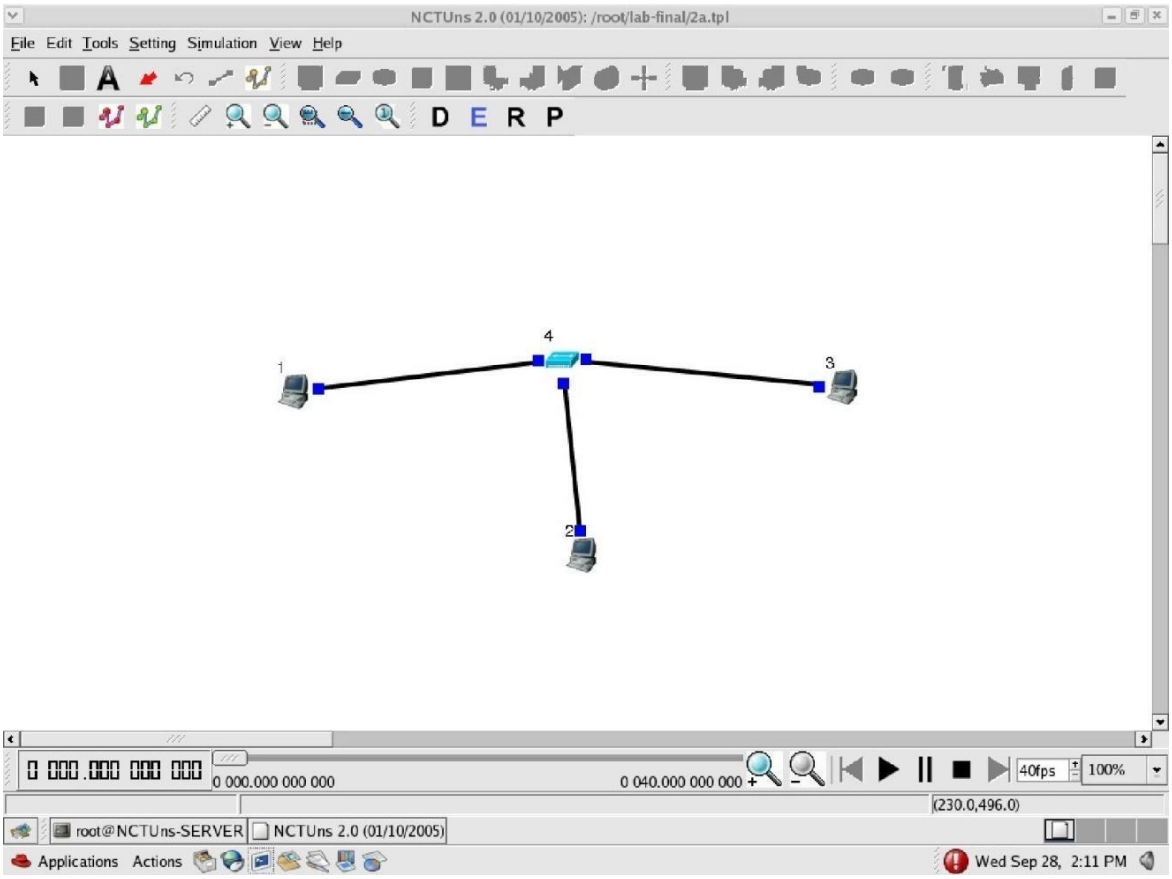
stg [-u payload size duration] [Host address]
rtg [-u] [-w log]

Step 5: Set the output throughput log to determine the number of packets sent by TCP/UDP as described in experiment 1.

Step 6: To view the results, go to the filename.results folder.

The screenshot of the topology is shown below:

OUTPUT:


NCTUns 2.0 (01/10/2005): /root/lab-final/2a.tpl
File Edit Tools Setting Simulation View Help
D E R P
4
1
3
2
0 000.000 000 000
0 040.000 000 000
40fps
100%
(230.0,496.0)
root@NCTUns-SERVER
NCTUns 2.0 (01/10/2005)
Applications Actions
Wed Sep 28, 2:11 PM

16 .Simulate the transmission of ping messages over a network topology consisting of 6 nodes and find the number of packets dropped due to congestion.

STEPS:

Step 1: Click on the subnet icon on the toolbar and then click on the screen of the working window.

Step 2: Select the required number of host sand a suitable radius between the host and the switch.

Step 3: In the edit mode, get the IP address of one of the hosts say, host 1 and then for the other host say, host2 set the drop packet and no: of collisions statistics as described in the earlier experiments.

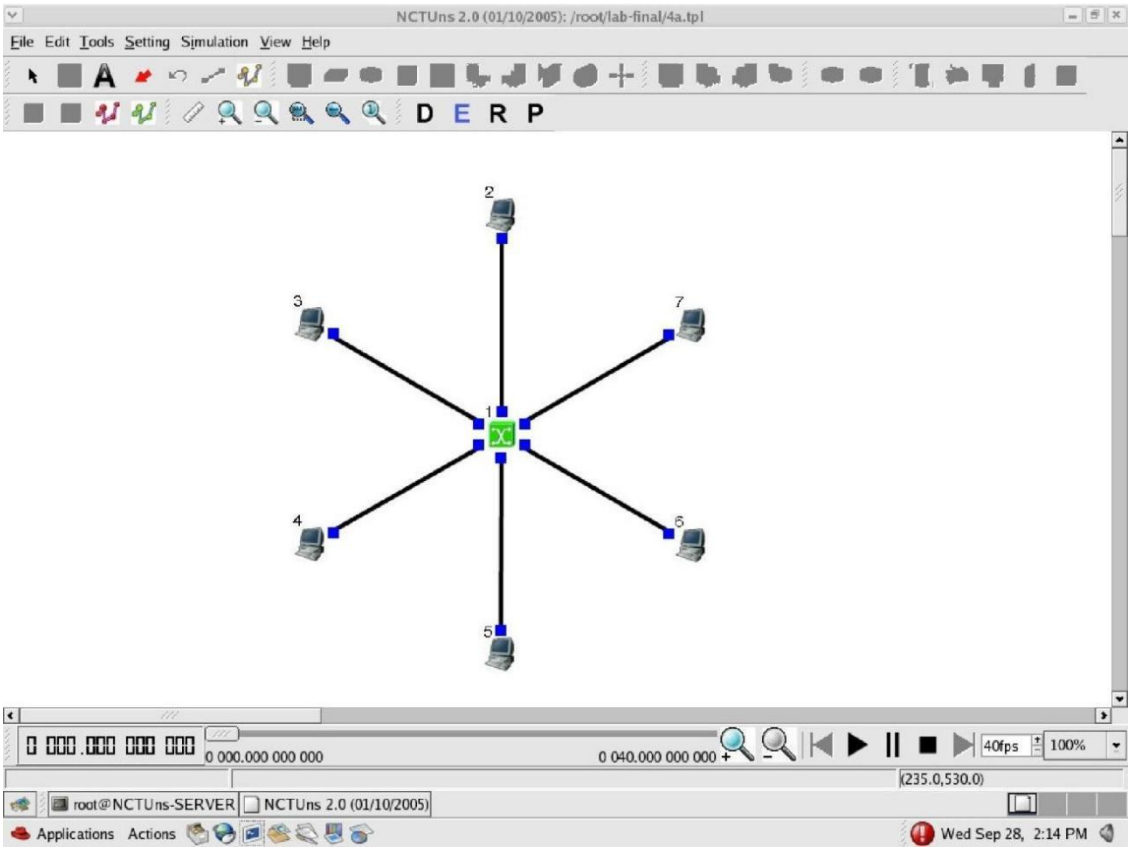
Step 4: Now run the simulation.

Step 5: Now click on any one of the hosts and click on command console and ping the destination node. ping IP Address of the host

Note: The no: of drop packets are obtained only when the traffic is more in the network. For checking the no of packets dropped press ctrl+C

The screenshot of the topology is shown below:

OUTPUT:


NCTUns 2.0 (01/10/2005): /root/lab-final/4a.tpl
File Edit Tools Setting Simulation View Help
D E R P
1
2
3
4
5
6
7
0 000.000 000 000
0 040.000 000 000
40fps
100%
(235.0,530.0)
root@NCTUns-SERVER
NCTUns 2.0 (01/10/2005)
Applications Actions
Wed Sep 28, 2:14 PM

17. Simulate an Ethernet LAN using N nodes (6-10), change error rate and data rate and compare throughput.

STEPS:

Step 1: Connect one set of hosts with a hub and another set of hosts also through a hub and connect these two hubs through a switch. This forms an EthernetLAN.

Step 2: Setup a TCP connection between a host on one hub and host on another hub using the following command:
stcp [-p port] [-l writesize] hostIPaddr rtcp [-p port] [-l readsize]

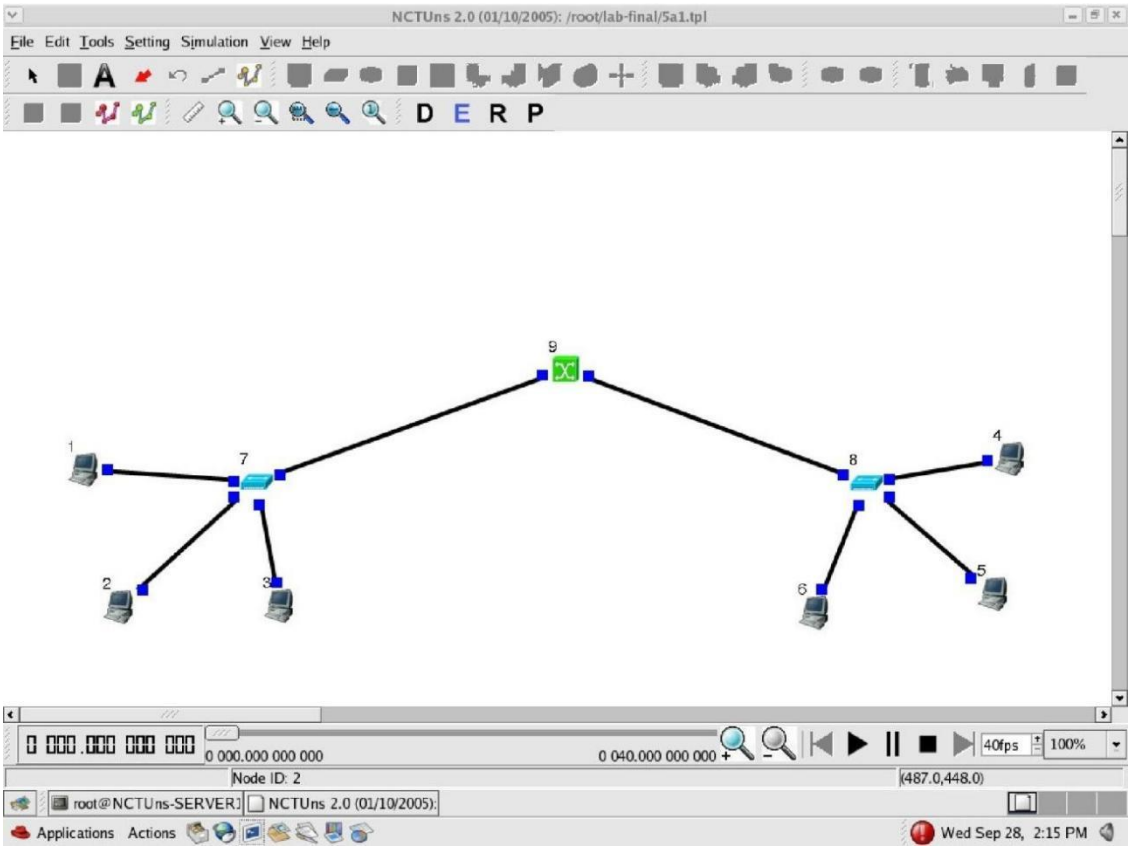
Step 3: Setup the error rate, data rate in the physical layer, input and output throughput in the mac layer as described above.

Step 4: Change error rate and data rate and compare the throughputs.

Step 5: View the results in the filename.results.

The screenshot of the topology is shown below:

OUTPUT:


NCTUns 2.0 (01/10/2005): /root/lab-final/5a1.tpl
File Edit Tools Setting Simulation View Help
D E R P
9
1
7
8
4
2
3
6
5
0 000.000 000 000
0 040.000 000 000
40fps
100%
Node ID: 2
(487.0,448.0)
root@NCTUns-SERVER:
NCTUns 2.0 (01/10/2005):
Applications Actions
Wed Sep 28, 2:15 PM

18.Simulate an Ethernet LAN using N nodes and set multiple traffic nodes and plot congestion window for different source/destination.* ns2/ns3/CISCO Packet Tracet/OPNET/any other network simulator may be used for simulation experiments.

STEPS:

Step 1: Connect one set of hosts with a hub and another set of hosts also through a hub and connect these two hubs through a switch. This forms an Ethernet LAN.

Step 2: Setup multiple traffic connections between the hosts on one hub and hosts on another hub using the following command:
stcp [-p port] [-l writesize] hostIPaddr rtcp [-p port] [-l readsize]

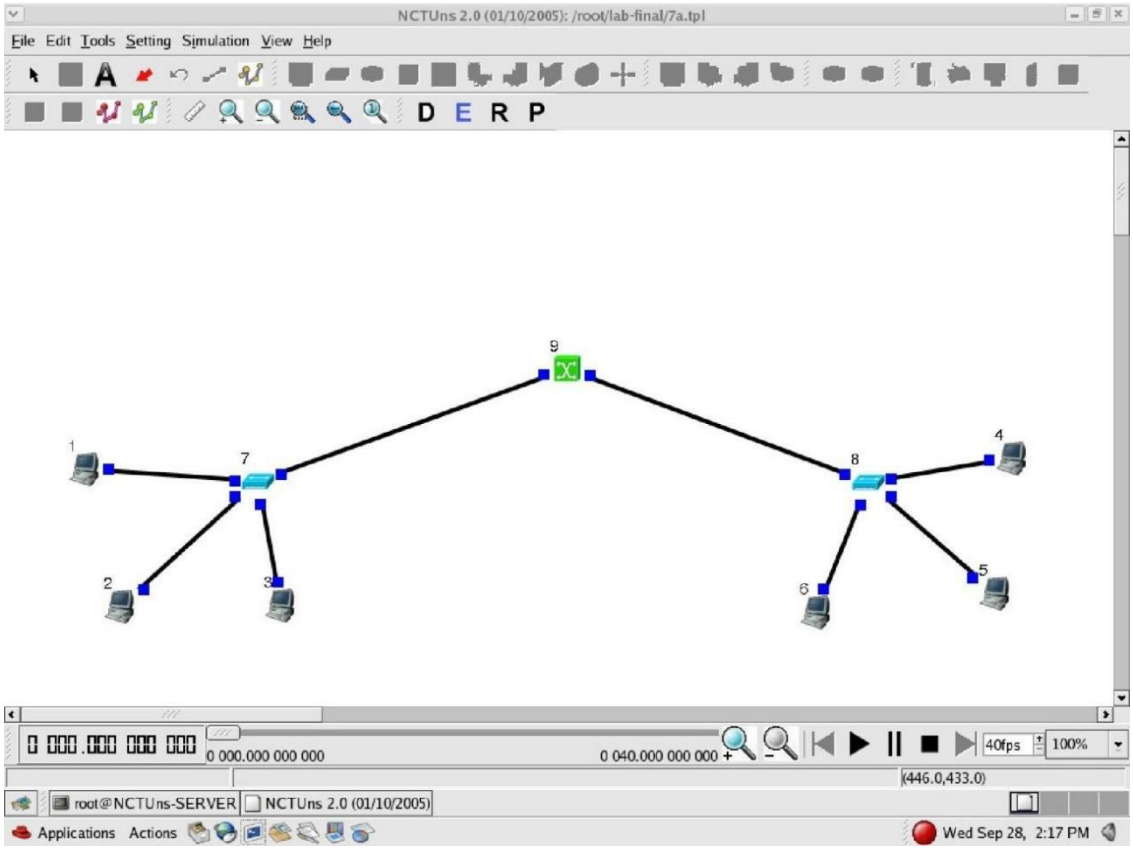
Step 3: Setup the collision log at the destination hosts in the MAC layer as described in the earlier experiments.

Step 4: To plot the congestion window go to Menu->Tools->Plot Graph->File->open->filename.results->filename.coll.log

Step 5: View the results in the filename.results.

The screenshot of the topology is shown below:

OUTPUT:


NCTUns 2.0 (01/10/2005): /root/lab-final/7a.tpl
File Edit Tools Setting Simulation View Help
D E R P
9
1
7
8
4
2
3
6
5
0 000.000 000 000
0 040.000 000 000
40fps
100%
(446.0,433.0)
root@NCTUns-SERVER
NCTUns 2.0 (01/10/2005)
Applications Actions
Wed Sep 28, 2:17 PM